

GameController3 完整指南

目录

1. 获取并编辑GameController3
2. 2025规则更新要点
3. GC模式选择说明
4. 比赛流程详解
5. 基于gc更新的关键代码解析
6. 三种出界状态详解

1、获取并编辑GameController3:

1.1 获取源码:

```
# 克隆官方仓库
git clone https://github.com/RoboCup-SPL/GameController3.git
cd GameController3
```

1.2 环境要求

根据 GameController3/README.md，需要安装以下依赖：

- **Rust** (建议1.77.2或更新版本)
- **Node.js 和 npm**
- **libclang** (用于bindgen)
- **Tauri**依赖 (参考 <https://tauri.app/start/prerequisites/>)

1.3 安装依赖

1.3.1 安装 Node.js 和 npm

```
# 方式1: 使用apt安装
sudo apt update
sudo apt install nodejs npm
# 方式2: 使用nvm安装 (推荐, 可以管理多版本)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh
source ~/.bashrc
nvm install --lts
nvm use --lts
# 验证安装
```

```
node --version  
npm --version
```

1.3.2 安装 libclang

```
sudo apt install libclang-dev
```

1.3.3 安装 Tauri 依赖

根据 Tauri 官方文档 (<https://tauri.app/start/prerequisites/>), Ubuntu/Debian 需要安装:

```
sudo apt update  
sudo apt install libwebkit2gtk-4.1-dev \  
build-essential \  
curl \  
wget \  
file \  
libxdo-dev \  
libssl-dev \  
libayatana-appindicator3-dev \  
libsvg2-dev
```

1.3.4 安装/升级 Rust

如果系统 Rust 版本过低或未安装, 使用 rustup:

```
# 安装rustup (如果没有)  
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s --  
  
# 加载环境变量  
source "$HOME/.cargo/env"  
  
# 验证版本  
rustc --version # 应该 >= 1.77.2  
  
# 如果版本过低, 升级Rust  
rustup update stable
```

1.4编译步骤

步骤1: 安装tauri-cli(开发模式需要)

```
cargo install tauri-cli
```

步骤2：编译前端

```
cd frontend
npm ci                #安装依赖
npm run build         #编译前端
cd ..
```

步骤3：编译后端

```
cargo build -r          #-r      表示release模式
```

1.5运行GameController

```
#方式1：通过cargo运行：
cargo run -r
#方式2：直接运行编译好的二进制文件或进入该目录
./tartet/release/gamer_controller_app
```

2. 2025规则更新要点(部分更新的规则)

根据2025-6月发布的规则文件，主要变更如下 (**该部分只适用2025年更新规则)：

2.1 Ready信号延迟调整

- Champions Cup：延迟增加到 45秒
- Challenge Shield：延迟增加到 35秒
- Ready视觉信号现在也适用于Challenge Shield

2.2 Free Kick踢球队伍通知方式（最重要变更）

- **Champions Cup模式**：free kick的踢球队伍通过**裁判视觉手势**通知
- **GameController**不再向机器人发送踢球队伍信息
- 机器人必须识别裁判手势来判断哪队踢球

2.3 Standby状态下的Motion处罚

- 错误检测Ready信号导致的Motion处罚变成**全队处罚**
- 处罚时间缩短

2.4 团队通信消息预算

- Standby状态下发送的通信数据包会**消耗**消息预算

2.5 未执行Free Kick的后果

- 如果一支队伍未能执行free kick，对方可以**直接尝试进球**

2.6 间接踢球规则扩展

- 2024年Champions Cup的间接踢球规则现在也适用于Challenge Shield
- Challenge Shield队伍可使用fallback选项

2.7 处罚可提前取消

- 裁判错误给出的处罚可以提前取消

2.8 球停止规则

- 每半场可稍微延长，让进攻局面完成

3.GC模式选择说明

3.1 四种模式对比

| 模式 | 机器人数量 | hideKickingSide | 需要识别裁判手势？ | 适用场景 |

|-----|-----|-----|-----|-----|

| Champions Cup | 7人 | true | ☒ 需要 | 正式比赛(7v7) |

| Champions Cup 5 vs. 5 | 5人 | true | ☒ 需要 | 正式比赛(5v5) |

| Challenge Shield | 5人 | false | ❌ 不需要 | 新队伍/入门级 |

| Most Passes Leaderboard | 2人 | false | ❌ 不需要 | 技术挑战赛 |

3.2 配置文件位置

- Champions Cup: GameController3/config/champions_cup/params.yaml
- Champions Cup 5v5(我们参加这组):
GameController3/config/champions_cup_5/params.yaml
- Challenge Shield: GameController3/config/challenge_shield/params.yaml

3.3 关键配置项

```
# Champions Cup / Champions Cup 5v5 (我们是这组)
hideKickingSide: true # 隐藏踢球队伍信息
# Challenge Shield
hideKickingSide: false # 不隐藏踢球队伍信息
```

4. 比赛流程详解（2026年比赛）

4.1 出界处理流程（以Kick-in为例）

GC操作员视角

1. 裁判看到球出界，喊“Kick-in blue”(蓝队踢)
2. GC操作员听到后，点击**蓝队下方**的“Kick-in”按钮
3. GC内部记录：kicking_side = Some(蓝队)
4. GC发送数据包给机器人

机器人视角（Champions Cup模式）

1. 收到数据包：
 - setPlay = 4 (SET_PLAY_KICK_IN) → 知道是Kick-in
 - kickingTeam = 255 (KICKING_TEAM_NONE) → 不知道哪队踢球

2. 机器人需要：

- 找到裁判
- 识别裁判举起的手臂
- 判断手臂指向哪个球门
- 从而确定哪队踢球

机器人视角（Challenge Shield模式）（简要说明，我们不参加）

1. 收到数据包：

- `setPlay = 4` (`SET_PLAY_KICK_IN`) → 知道是Kick-in
- `kickingTeam = 5` (队伍编号) → 直接知道哪队踢球

2. 不需要识别裁判手势

4.2 裁判手势说明

根据规则3.9.1节：

“The gesture consists of raising one arm horizontally and sideways, parallel to the field line... The raised arm is the one pointing towards the defending team’s goal”

- 裁判举起一只手臂，水平向侧面伸出，与场地线平行
- 举起的手臂指向**防守方的球门**（即不踢球的那队的球门）
- 手势保持至少5秒

5. 关键代码解析

5.1 核心文件位置

```
GameController3/  
├─ game_controller_msgs/  
│ └─ src/  
│   └─ control_message.rs # 发送给机器人的数据包构建  
│   └─ headers/  
│   └─ RoboCupGameControlData.h # 数据包结构定义和常量  
└─ game_controller_core/
```

```

| └─ src/
| └─ actions/
| └─ start_set_play.rs # Set Play动作的执行和合法性检查
└─ config/
└─ champions_cup/params.yaml # Champions Cup配置
└─ champions_cup_5/params.yaml # Champions Cup 5v5配置
└─ challenge_shield/params.yaml # Challenge Shield配置

```

5.2 常量定义

文件: GameController3/game_controller_msgs/headers/RoboCupGameControlData.h

```

// Set Play 类型常量 (第45-50行)
#define SET_PLAY_NONE 0 // 无 set play
#define SET_PLAY_GOAL_KICK 1 // Goal Kick (球门球)
#define SET_PLAY_PUSHING_FREE_KICK 2 // Pushing Free Kick
#define SET_PLAY_CORNER_KICK 3 // Corner Kick (角球)
#define SET_PLAY_KICK_IN 4 // Kick-in (边线球)
#define SET_PLAY_PENALTY_KICK 5 // Penalty Kick (点球)
// 踢球队伍常量 (第51行)
#define KICKING_TEAM_NONE 255 // 未知/隐藏

```

数据包结构 (第98-104行) :

```

struct RoboCupGameControlData {
// ... 其他字段 ...
uint8_t setPlay; // 当前 set play 类型 (0-5)
uint8_t firstHalf; // 是否上半场
uint8_t kickingTeam; // 踢球队伍编号, 或255表示未知
int16_t secsRemaining; // 剩余秒数
int16_t secondaryTime; // 次要计时器 (如free kick倒计时)
// ... 其他字段 ...
};

```

5.3 hide_kicking_side 判断逻辑

文件: GameController3/game_controller_msgs/src/control_message.rs

位置: 第163-170行

```

let hide_kicking_side = !to_monitor // 条件1: 不是发给监控程序
&& params.competition.hide_kicking_side // 条件2: 配置文件中 hideKickingSide:

```

```
&& game.phase != Phase::PenaltyShootout // 条件3: 不是点球大战
&& ((game.state == State::Ready
|| game.state == State::Set
|| game.state == State::Playing) // 条件4: 在 Ready/Set/Playing 状态
&& game.set_play != SetPlay::KickOff); // 条件5: 不是开球
```

解释: 当以上5个条件**全部满足**时, `hide_kicking_side = true`, 机器人将收不到踢球队伍信息。

5.4 set_play 字段设置(与5.2节结合)

文件: GameController3/game_controller_msgs/src/control_message.rs

位置: 第193-200行

```
set_play: match game.set_play {
  SetPlay::NoSetPlay | SetPlay::KickOff => SET_PLAY_NONE, // 无/开球 → 0
  SetPlay::KickIn => SET_PLAY_KICK_IN, // Kick-in → 4
  SetPlay::GoalKick => SET_PLAY_GOAL_KICK, // Goal Kick → 1
  SetPlay::CornerKick => SET_PLAY_CORNER_KICK, // Corner Kick → 3
  SetPlay::PushingFreeKick => SET_PLAY_PUSHING_FREE_KICK, // Pushing Free Kic
  SetPlay::PenaltyKick => SET_PLAY_PENALTY_KICK, // Penalty Kick → 5
},
```

解释: 这里将内部枚举转换为发送给机器人的数字常量。机器人**总是能收到**这个信息。

5.5 kicking_team 字段设置 (★最关键)

文件: GameController3/game_controller_msgs/src/control_message.rs

位置: 第202-204行

```
kicking_team: game
.kicking_side // 获取GC内部记录的踢球队伍
.filter(|_| !hide_kicking_side) // 如果hide_kicking_side=true, 则过滤掉
.map_or(KICKING_TEAM_NONE, |side| params.game.teams[side].number),
```

解释:

1. `game.kicking_side` - GC内部记录的踢球队伍 (Some(Home)或Some(Away))

- 2. `.filter(|_| !hide_kicking_side)` - 如果 `hide_kicking_side = true` , 返回 `None`
- 3. `.map_or(KICKING_TEAM_NONE, ...)` - 如果是 `None` , 返回255; 否则返回队伍编号

6. 三种出界状态详解

6.1 Kick-in（边线出界）—— 需要识别裁判手势

6.1.1 触发条件

球越过边线（touchline），无论是哪支队伍踢出去的。

6.1.2 球的位置

裁判将球放在球出界的边线位置。

6.1.3 GC操作员操作流程

- 1. 裁判观察是哪支队伍最后触球，喊出“Kick-in <颜色>”(如“Kick-in blue”)
- 2. GC操作员听到后，点击**获得踢球权的队伍**下方的“Kick-in”按钮
- 3. GC内部记录 `game.kicking_side = Some(该队伍)`
- 4. GC构建数据包并发送给机器人

6.1.4 机器人收到的数据包

字段	Champions Cup	Challenge Shield
setPlay	4 (SET_PLAY_KICK_IN)	4 (SET_PLAY_KICK_IN)
kickingTeam	255 (未知)	队伍编号
secondaryTime	30秒倒计时	30秒倒计时

6.1.5 ★ 机器人判断踢球队伍的方式（Champions Cup模式）

由于 kickingTeam = 255，机器人**必须通过识别裁判手势**来判断哪队踢球：

B-Human 机器人端代码：

文件位置：

Src/Modules/Infrastructure/GameStateProvider/GameStateProvider.cpp

第861-879行 - 通过裁判手势判断Kick-in踢球队伍：

```
void GameStateProvider::updateKickingTeamByRefereeSignal(GameState& gameSt

{
    // Only execute during free kick when the kicking team is hidden.
    // 只在free kick且踢球队伍被隐藏时执行
    if(gameState.kickingTeamKnown || !gameState.isKickIn())
        return;

    // 检测裁判手势：leftHandTeam表示左边队伍，根据手势方向判断
    // kickInRight = 裁判手臂指向右边球门 → 左边队伍踢球
    // kickInLeft = 裁判手臂指向左边球门 → 右边队伍踢球
    const bool forOwnTeam = checkForRefereeSignal(gameState,
gameState.leftHandTeam ? RefereeSignal::kickInRight : RefereeSignal::kickIn
    const bool forOpponentTeam = checkForRefereeSignal(gameState,
gameState.leftHandTeam ? RefereeSignal::kickInLeft : RefereeSignal::kickInR
    if(forOwnTeam)
    {
        isKickingTeam = true; // 我方踢球
        gameState.state = GameState::ownKickIn; // 更新状态为我方Kick-in
    }
    if(forOpponentTeam) // or for both teams
    {
        isKickingTeam = false; // 对方踢球
        gameState.state = GameState::opponentKickIn; // 更新状态为对方Kick-in
    }
}
```

第991-997行 - 处理GC数据包中的Kick-in：

```
else if(gameControllerData.setPlay == SET_PLAY_KICK_IN)
{
    // 如果kickingTeam无效(=255)且不是之前的Kick-in状态，默认假设对方踢球
    if(!isKickingTeamValid && !GameState::isKickIn(lastGameControllerState))
        isKickingTeam = false; // 默认假设对方踢球，等待裁判手势确认
    return isKickingTeam ? GameState::ownKickIn : GameState::opponentKickIn;
}
```


2. GC操作员点击**防守方（获得踢球权）**下方的“Goal Kick”按钮
3. 裁判将球放在球门区角落

6.2.4 机器人收到的数据包

字段	Champions Cup	Challenge Shield
setPlay	1 (SET_PLAY_GOAL_KICK)	1 (SET_PLAY_GOAL_KICK)
kickingTeam	255 (未知)	队伍编号
secondaryTime	30秒倒计时	30秒倒计时

6.2.5 ★ 机器人判断踢球队伍的方式（Champions Cup模式）

Goal Kick的特点是：球一定放在某支队伍防守的球门区角落，该队伍获得踢球权。

机器人可以通过球的位置和球所在的半场来判断：

判断逻辑：

Goal Kick规则：

- 球放在【防守方】的球门区角落
- 【防守方】获得踢球权
- 因此：球在哪个半场的球门区，哪支队伍踢球

B-Human 机器人端代码：

文件位置：

Src/Modules/Infrastructure/GameStateProvider/GameStateProvider.cpp

第999-1008行 - 通过球位置判断Goal Kick踢球队伍：

```
else if(gameControllerData.setPlay == SET_PLAY_GOAL_KICK)
{
    if(!isKickingTeamValid) // kickingTeam == 255, 需要自己判断
    {
        if(!gameStateOverridden && theTeamBallModel.isValid)
        // 关键判断：球的x坐标 < 0 表示球在我方半场（我方防守的球门区）
        // 因此我方踢球 (isKickingTeam = true)
        isKickingTeam = theTeamBallModel.position.x() < 0;
        else if(!GameState::isGoalKick(lastGameControllerState))
            isKickingTeam = false; // 无法判断时默认对方踢球
    }
}
```

```

    }
    return isKickingTeam ? GameState::ownGoalKick : GameState::opponentGoal
  }
}

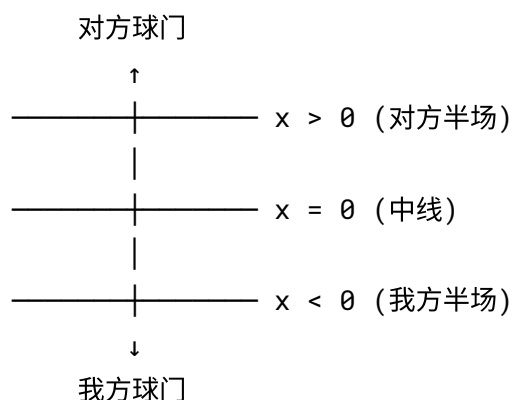
```

代码解释:

1. 机器人收到 `setPlay = SET_PLAY_GOAL_KICK` 且 `kickingTeam = 255`
2. 检查 `theTeamBallModel.isValid` (球位置是否有效)
3. **关键判断:** `theTeamBallModel.position.x() < 0`
 - $x < 0$: 球在我方半场 (我方防守的球门区) → **我方踢球**
 - $x > 0$: 球在对方半场 (对方防守的球门区) → **对方踢球**
4. 根据判断结果返回 `ownGoalKick` 或 `opponentGoalKick`

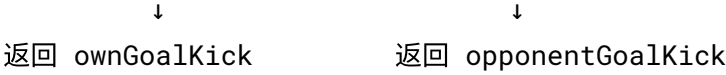
坐标系说明:

B-Human坐标系 (从我方视角):



判断流程图:





示例场景：

假设：我方防守左球门（ $x < 0$ 的一侧），对方防守右球门（ $x > 0$ 的一侧）
场景：对方进攻，将球踢出我方的球门线
结果：
- 裁判喊 "Goal Kick"
- 球放在我方半场的球门区角落
- `theTeamBallModel.position.x() < 0`
- `isKickingTeam = true` → 我方踢球
- 返回 `GameState::ownGoalKick`

6.3 Corner Kick（角球）—— 根据球位置判断

6.3.1 触发条件

球越过球门线（goal line），且最后触球的是**防守方**（即守门员或后卫将球踢出自己的球门线）。

6.3.2 球的位置

裁判将球放在**角球点**（corner），即球门线与边线的交点。

6.3.3 GC操作员操作流程

- 1. 裁判观察到防守方将球踢出球门线，喊出 "Corner Kick <颜色>" (如 "Corner Kick blue")
- 2. GC操作员点击****进攻方（获得踢球权）****下方的 "Corner Kick" 按钮
- 3. 裁判将球放在角球点

6.3.4 机器人收到的数据包

字段	Champions Cup	Challenge Shield
setPlay	3 (SET_PLAY_CORNER_KICK)	3 (SET_PLAY_CORNER_KICK)
kickingTeam	255 (未知)	队伍编号

字段	Champions Cup	Challenge Shield
secondaryTime	30秒倒计时	30秒倒计时

6.3.5 ★ 机器人判断踢球队伍的方式（Champions Cup模式）

Corner Kick的特点是：球放在某支队伍防守的半场角落，但由对方（进攻方）踢球。

这与Goal Kick相反：

- Goal Kick：球在哪个半场，该半场的防守队伍踢球
- Corner Kick：球在哪个半场，**对方队伍（进攻方）**踢球

判断逻辑：

Corner Kick规则：

- 球放在【防守方】的半场角落
- 【进攻方】获得踢球权
- 因此：球在哪个半场的角落，【另一支队伍】踢球

B-Human 机器人端实际代码：

文件位置：

Src/Modules/Infrastructure/GameStateProvider/GameStateProvider.cpp

第1009-1019行 - 通过球位置判断Corner Kick踢球队伍：

```

else if(gameControllerData.setPlay == SET_PLAY_CORNER_KICK)
{
    if(!isKickingTeamValid) // kickingTeam == 255, 需要自己判断
    {
        if(!gameStateOverridden && theTeamBallModel.isValid)
        // 关键判断：球的x坐标 > 0 表示球在对方半场（对方防守的角落）
        // Corner Kick由进攻方踢，所以球在对方半场时我方踢球
        isKickingTeam = theTeamBallModel.position.x() > 0;
        else if(!GameState::isCornerKick(lastGameControllerState))
            isKickingTeam = false; // 无法判断时默认对方踢球
    }
    return isKickingTeam ? GameState::ownCornerKick : GameState::opponentCo
}

```

代码解释：

1. 机器人收到 `setPlay = SET_PLAY_CORNER_KICK` 且 `kickingTeam = 255`
2. 检查 `theTeamBallModel.isValid` (球位置是否有效)
3. **关键判断**: `theTeamBallModel.position.x() > 0` (注意: 与Goal Kick相反!)
 - $x > 0$: 球在对方半场 (对方防守的角落) → **我方踢球** (我方是进攻方)
 - $x < 0$: 球在我方半场 (我方防守的角落) → **对方踢球** (对方是进攻方)
4. 根据判断结果返回 `ownCornerKick` 或 `opponentCornerKick`

Goal Kick vs Corner Kick 判断对比：

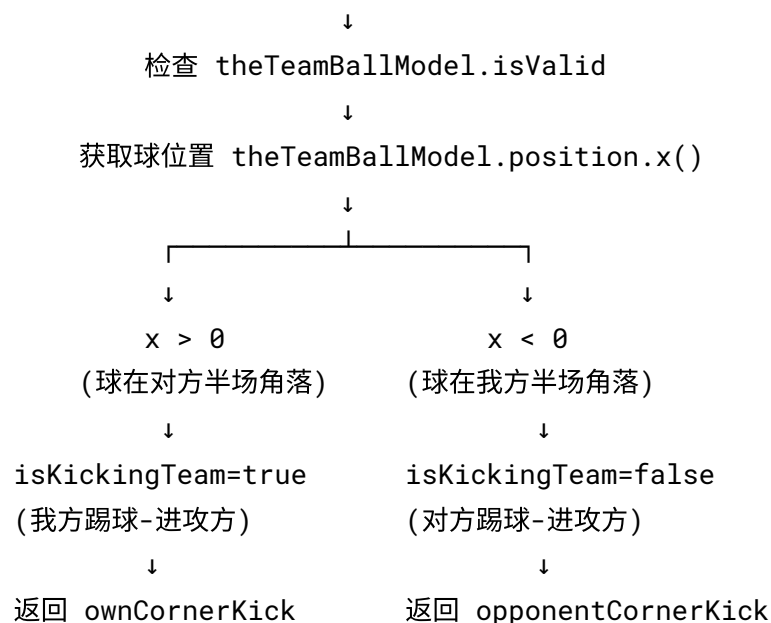
```
// Goal Kick: 球在我方半场 → 我方踢球 (防守方踢)
isKickingTeam = theTeamBallModel.position.x() < 0
```

```
// Corner Kick: 球在对方半场 → 我方踢球 (进攻方踢)
```

```
isKickingTeam = theTeamBallModel.position.x() > 0;
```

判断流程图：

机器人收到 `setPlay=3` (Corner Kick), `kickingTeam=255`



示例场景：

假设：我方防守左球门 ($x < 0$ 的一侧)，对方防守右球门 ($x > 0$ 的一侧)

场景：对方守门员将球踢出自己的球门线 (右侧)

结果：

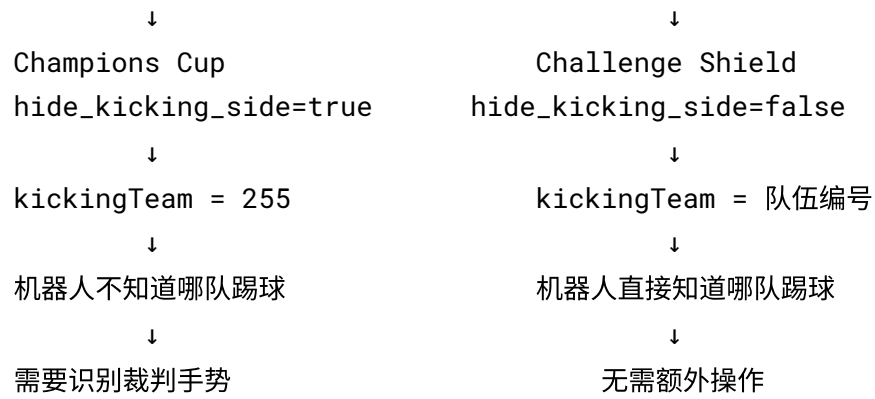
- 裁判喊 "Corner Kick"
- 球放在对方半场的角球点
- `theTeamBallModel.position.x() > 0`
- `isKickingTeam = true` → 我方踢球（我方是进攻方）
- 返回 `GameState::ownCornerKick`

三种出界状态判断方式总结：

出界类型	判断方式	关键代码
Kick-in	识别裁判手势	<code>checkForRefereeSignal(RefereeSignal::kickInLeft/Right)</code>
Goal Kick	球位置 $x < 0$ → 我方踢	<code>isKickingTeam = theTeamBallModel.position.x() < 0</code>
Corner Kick	球位置 $x > 0$ → 我方踢	<code>isKickingTeam = theTeamBallModel.position.x() > 0</code>

附录：代码流程图





参考资料

- 官方GameController仓库: <https://github.com/RoboCup-SPL/GameController3>
- B-Human 2025 Changelog: <https://docs.b-human.de/master/changelog/2025/>
- RoboCup SPL 2025规则文档